

Credit Name: CSE 3130 Object-Oriented Programming 2.
Assignment Name: UEmployee

The following questions can help you in thinking critically about your problem-solving processes:

Understanding the Problem:

How did you approach understanding the challenge?

- Basically I visualize the project looking like taking attendance and checking information for employees.
- So the first idea I have is to simplify the project by imaging. I am doing a simple attendance-taking project like I did for students before.

Were there any parts of the problem you found confusing at first? If so, how did you resolve that confusion?

- This is the first project that I attempted and by combining multiple classes together, without guidance like the SkillBuilders so it is a little confusing for me to work on the project.
- So that I recall the steps and methods in the textbook also with the demo on Github to guide me how each method works for and how parenting works when I call for variables in the parent classes.

Planning the Solution:

Did you create any or break the problem into smaller steps before coding?

- I tend to make it easy by just doing it step by step and checking it before I move on so that I can keep track of the bugs and fix it instantly if there are any.

How did you decide on the tools, data structures, or algorithms to use?

- So I created the UEmployee class like what the prompt asks for.
- Then I implemented the data from the UEmployee and continue working on the Staff and Faculty by adding the information the prompt needs (workplace for Faculty and position in Staff)

Implementation:

Did you write the code in small or attempt the entire solution at first?

- I broke down the project in this order
 - UEmployee

```
private String name; //Stores the employee's name//
private double salary; //Stores the employee's salary//

//Constructor: initializes the employee's name and salary with the provided values//
public UEmployee(String n, double s) {
    name = n; //Sets the employee's name//
    salary = s; //Sets the employee's salary//
}
```

- I created two variables to store the information about the name and salary of the worker and declare the format inside the constructor. The UEmployee constructor will require name and salary to build the basic information for the user

```
//Returns the employee's name
public String getName() {
    return name;
}

//Returns the employee's salary
public double getSalary() {
    return salary;
}
```

- Then I made two more classes getName() and getSalary() to return their information.
- I will use these two in the Faculty and Staff classes since UEmployee is their parent class.

○ Staff

```
public Staff(String name, double salary) {
    super(name,salary); //Calls UEmployee's
}
```

- First I called the variables from the parent class, which I had declared in UEmployee includes name and salary.

```
public void setStaff(String position) {
    title = position;
}
```

- Then I added one more setStaff public void class, this is used to collect the name of their position.

```
public String getStaff() {
    String position = getName() + ", income $" + getSalary() + " currently doing " + title;
    return position;
}
```

- Then I used the getName() and getSalary() method in UEmployee to finalize the updated getStaff() method.
- This will introduce or use to double check their name, salary and position are matched to what the user has put in.

○ Faculty:

```
public Faculty(String name, double salary) {
    super(name,salary); //Calls UEmployee's
}
```

- First I called the variables from the parent class, which I had declared in UEmployee includes name and salary.

```
public void setFaculty(String w) {
    workplace = w;
}
```

- Then I added one more setFaculty public void class, this is used to collect the name of their workplace.

```
public String getFaculty() {
    String work = getName() + ", income $" + getSalary() + " currently working at " + workplace;
    return work;
}
```

- Then I used the getName() and getSalary() method in UEmployee to finalize the updated getFaculty() method.

- This will introduce or use to double check their name, salary and workplace are matched to what the user has put in.
- Test

```
do {
    System.out.print("Enter your name: ");
    name = userInput.nextLine(); //Reads the user's full name including spaces//

    System.out.print("Enter your current salary: $");
    salaryInput = userInput.nextLine(); //Reads salary as a String so it can be validated before converting//
}
```

- First I collected the user name and salary

```
try {

    salary = Double.parseDouble(salaryInput); //Attempts to convert the salary String to a double//
    /**
     * If statement
     */
    if (!name.isBlank() && salary > 0) { //Checks that name is not empty and salary is a positive number//
        i = false; //Valid input received, exit the loop//
    }
    else if (!(name instanceof String) || !((Object)salary) instanceof Double) { //Checks that both fields are of the correct type//
        System.out.print("Invalid. Please try again.");
    }
}
catch (NumberFormatException e) { //Catches the case where the salary input cannot be parsed as a number//
    System.out.println("Invalid salary. Please enter a numeric value.");
    System.out.println("-----");
}
```

- Then I threw the user inputs into the try-catch, so it can catch when the user enters the wrong type of data or leaves any input collector empty.
- All of the process of entering name and salary is put inside the do-while loop and it only stops when the user enters the correct information needed.

```
//Creates a base UEmployee object to display the validated name and salary//
UEmployee employee = new UEmployee(name,salary);
System.out.println("-----");
System.out.println("User's information taken successfully");
System.out.println("Name: " + employee.getName()); //Displays the employee's name//
System.out.println("Salary: $" + employee.getSalary()); //Displays the employee's salary//
System.out.println("-----");
```

- This step is where I first show them the basic information, which is where I test the project first so that I can make sure that the entire thing is work.

```
System.out.print("Enter your faculty: ");
String faculty = userInput.nextLine(); //Reads the faculty name//
Faculty employee_1 = new Faculty(name,salary); //Creates a Faculty object//
employee_1.setFaculty(faculty); //Sets the faculty member's faculty//

System.out.print("Enter your position: ");
String title = userInput.nextLine(); //Reads the staff title//
Staff employee_1 = new Staff(name,salary); //Creates a Staff object//
employee_1.setStaff(title); //Sets the staff member's position//
```

- Then I also collected the extended information which is the position and faculty they are currently working for.

```
//Displays the extended information for both Faculty and Staff roles//
System.out.println("Staff's information(Extended): \n"+
    "Faculty: " + employee_1.getFaculty()+"\n" //Prints faculty name, salary, and workplace//
    "Position: " + employee_1.getStaff()); //Prints staff name, salary, and job title//
```

- Finally, I show the user the finalized version of their information.

How did you test your solution along the way to make sure it was working?

- So first I tested only with the UEmployee class, I wanted to make sure that I can return the information.

- And since the Staff and Faculty are identical, I just copy-paste the same logic between those 2 classes and apply the same logic about the parent class.

Overcoming Challenges:

What part of the problem was the most difficult for you?

- I think the most difficult challenge for me is how I can check for the correct data type and information the user will put in the Test class.
- It took me the most time to come up with the algorithm to solve the problem.
 - First is that I did not realize that double cannot be used with the == to check for their values (To check is there any String value instead of double)
 - Then I changed the salary variable into String so that I can check for any String value inside the salary variable. However, I then realized that by that way, I could not check for any negative values.
 - For that reason I changed the salary back to double and then compared it with > (Basically when salary > 0) then it is the correct value. Then I found out instanceof Double to compare whether the salary is actually double or not. And if not, the computer now can return the notification that prompts the user to enter the correct data type.

Learning:

Was there anything you learned that you think will help you with future challenges?

- This is a good beginning to an actual project that requires parenting and calls variables from a class to a different class, by that I now understand how parenting works and allows me to move around information easier for my next projects (which is a good improvement when considering that I could not do the same thing in my SkillBuilders when I still need the Demo from Github and helps from Calen and Ihor).